



جامعة دمشق
كلية الهندسة
المعلوماتية
قسم الذكاء الصناعي

مشروع البرمجة التفرعية Ecommerce System

المهندس:

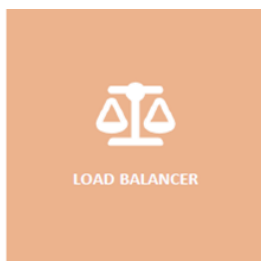
حذيفة عقيل

الطالبات:

البتول حسام صراميجو
بتول صهيب مستو
حلا عزو سليمان
حنين سامي قاطوع
سارة محمد غسان فيوم



Incoming Traffic



LOAD BALANCER



المشروع عبارة عن متجر الكتروني تم تطويره باستخدام:

Django, Django REST Framework, PostgreSQL

يهدف المشروع الى إدارة:

- المنتجات(Products)
 - الطلبات(Orders)
 - المستخدمين(Users)
 - المخزون(Stock Management)
- مع التركيز على: حماية البيانات من التضارب.

سبب اختيار Django:

- بنية جاهزة وسريعة.
- يحوي على نظام جاهز للمستخدمين.
- للتعامل مع قاعدة البيانات بسهولة عبر الـ ORM.
- دعم المعاملات لضمان سلامة العمليات باستخدام transaction.atomic().
- دعم التزامن.
- يدعم التعامل مع الطلبات المتزامنة.
- يوفر حلولاً مثل:
 - Atomic Updates باستخدام () F.
 - Row Locking باستخدام () select_for_update.
- يسهل بناء APIs بسهولة واحترافية.

تصميم النظام (System Design):

:Models

1. Product: الحقول (Name, stock, price).
2. Order: الحقول (User, status [PENDING, PAID, COMPLETED, CANCELLED], created_at).
3. OrderItem: الحقول (Order, Product, quantity).

APIs المنفذة:

1. المستخدمين: Register, login with JWT (Bearer token).
2. المنتجات: عرض كل المنتجات، عرض منتج واحد، اضافة منتج، تعديل منتج، حذف منتج.
3. الطلبات: * إنشاء طلب شراء (PENDING).
 - a. دفع الطلب (PAID).
 - b. إتمام الطلب.
 - c. الغاء الطلب (CANCELLED).

PostgreSQL أقوى من SQLite في locking, transactions لأنها تدعم العمليات المتزامنة.

الطلب الأول: حماية البيانات المشتركة من التضارب عند وصول عدة طلبات في نفس الوقت

المشكلة هي Race Condition، تحدث عندما عدة مستخدمين يطلبون نفس المنتج بنفس الوقت. مثال:

Stock = 2 ويوجد 3 طلبات بنفس الوقت، بدون حماية سوف ينجح ويقرأ جميع الطلبات بنجاح ويصبح - Stock = 1.

السبب أن العمليات تتم بشكل غير متزامن ولا يوجد قفل أو حماية على البيانات.

الحلول الممكنة:

1. Row locking using Select_for_update ()

يقوم بقفل الصف داخل قاعدة البيانات، مشكلته أنه أبطأ ويسبب انتظار.

2. Atomic Update Product.objects.filter(id = product_id, stock_gte = quantity).update(stock=F('stock') - quantity))

وهو الحل المستخدم في المشروع، تم استخدام Transaction.atomic() باستخدام transaction لضمان أن العملية كاملة تنجح أو تفشل باستخدام Atomic Update F() ، يتم التحقق من أن stock >= quantity ويتم التعديل مباشرة داخل قاعدة البيانات بدون قراءة القيمة أولاً.

والنتيجة: العملية تتم في خطوة واحدة لا يمكن لطلبين التعديل بنفس الوقت بشكل خاطئ يتم منع Race Condition.

🚦 السبب لاختيار هذا الحل لأنه أسرع، أبسط ، ويمنع التضارب بكفاءة عالية.

لاختبار هذا الطلب (اختبار التزامن):

لإرسال عدة طلبات بنفس الوقت تم استخدام سكربت والنتيجة كانت:

201 Created

201 Created

400 Not enough stock

أي أنه لم يتم بيع أكثر من الكمية المتاحة ولم يصبح المخزون سالب والنظام آمن من Race Condition.

الطلب الثاني: إدارة الموارد والتحكم في القدرة Capacity Control:

في هذا الجزء، ركزنا على هندسة الموارد الداخلية للنظام لضمان استقراره تحت الضغط العالي دون الحاجة لانتظار المهام الطويلة.

1. الية تجميع الخيوط Thread Pooling Strategy:

بدلاً من ترك المعالج يستهلك طاقته في فتح وإغلاق خيوط معالجة Threads بشكل عشوائي مع كل طلب، قمنا بحصر الموارد المتاحة للنظام:

تحديد القدرة الاستيعابية: استخدمنا `os.cpu_count()` لتحديد عدد الأنوية المتاحة في الجهاز وبناء عليه حددنا عدد العمال (Workers) وبالأستفادة من المعادلتين الآتيتين :

المهام المقيدة بالمعالج CPU-Bound Tasks:

المعادلة: $N + 1$

الشرح: نستخدم للمهام التي تستهلك المعالج بالكامل. نكتفي بهذا العدد لأن زيادة الخيوط عن عدد الأنوية لن تزيد السرعة، بل ستسبب بظناً نتيجة تبديل السياق Context Switching

المهام المقيدة بالإدخال والإخراج (I/O-Bound Tasks):

المعادلة: $N \times (1 + W/C)$

الشرح: حيث W زمن الانتظار و C زمن المعالجة تستخدم لعمليات قاعدة البيانات والشبكة وبما أن الخيوط تكون خاملة أثناء الانتظار يمكننا مضاعفة عددها مثلاً $5 \times N$ لاستغلال المعالج في مهام أخرى.

فصل المهام حسب النوع:

`IO_Worker` خصصناه للمهام التي تقضي معظم وقتها في انتظار قاعدة البيانات مثل `save` و `update` `CPU_Worker` للمهام التي تحتاج عمليات حسابية.

الفائدة الهندسية: منعنا النظام من الوصول لحالة الانهيار Crash نتيجة استهلاك الذاكرة المفرط إذا وصل الـ 1000 المستخدمين في لحظة واحدة.

```
19
20 n_cores = os.cpu_count() or 1
21
22 IO_WORKERS = n_cores * 5
23 io_executor = ThreadPoolExecutor(max_workers=IO_WORKERS, thread_name_prefix="IO_Worker")
24
25 CPU_WORKERS = n_cores + 1
26 cpu_executor = ThreadPoolExecutor(max_workers=CPU_WORKERS, thread_name_prefix="CPU_Worker")
27
```

2. محاكاة التحكم في الأحمال Capacity Simulation:

- الاستجابة الفورية Asynchronous Response في الـ register والـ Order Creation بمجرد التأكد من صحة البيانات (`is_valid`) يتم تفويض المهمة للـ Executor ليعمل في خيط منفصل وإعادة رد فوراً للمستخدم بحالة 202 Accepted هذا يمنع بقاء المستخدم في حالة انتظار ويحرر خيط المعالجة الرئيسي فوراً.
- التحكم بالزمن Time/Resource Simulation أضفنا `time.sleep(2)` في العمليات مثل الدفع `process_payment` لمحاكاة استهلاك المورد لفترة زمنية Latency ورغم هذا التأخير المتعمد، يبقى النظام قادراً على استقبال طلبات جديدة لأن المهمة تعمل في الخلفية الداخلية للتطبيق دون تعطيل المسار الرئيسي.

3. التخزين المؤقت الذكي Internal Caching:

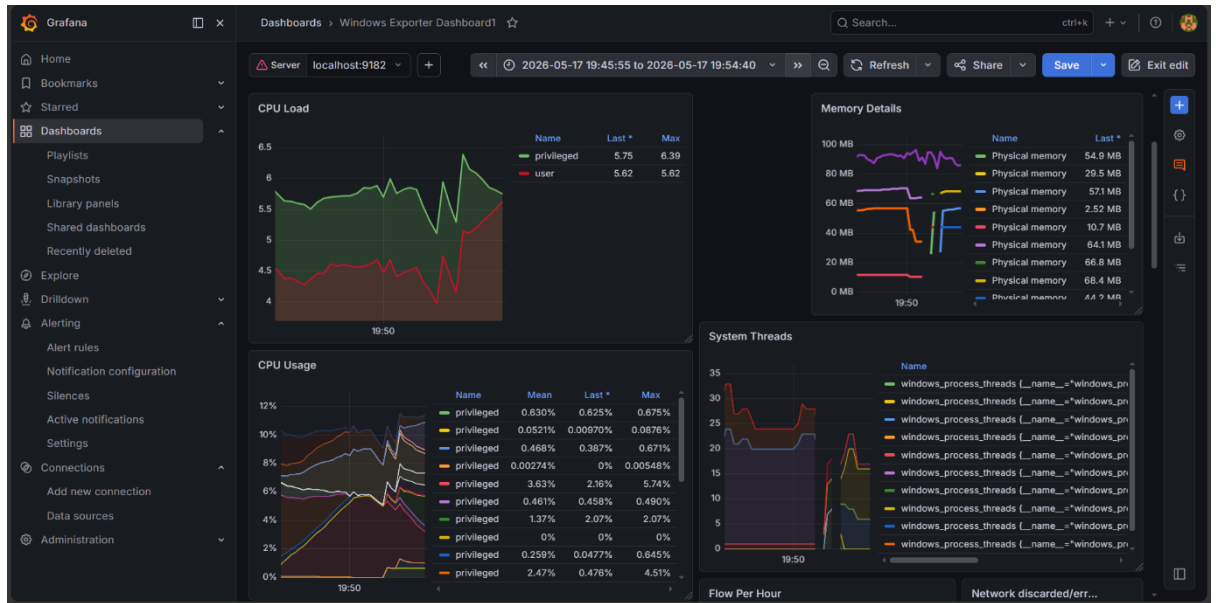
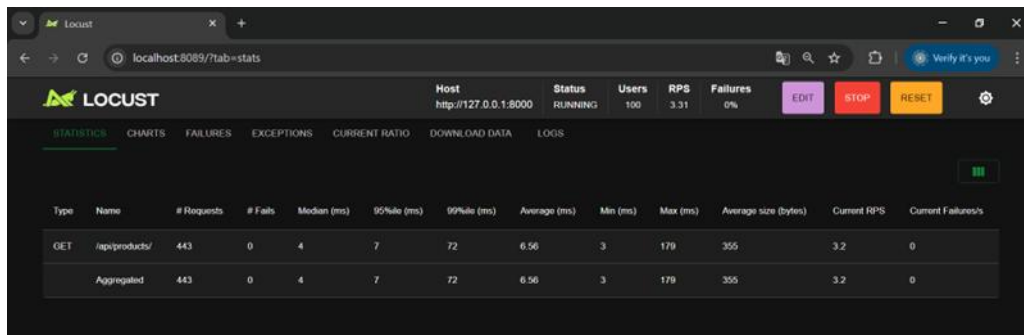
- تقليل العبء Database Offloading استخدمنا الكاش في `ProductListCreateAPIView` لتقليل الضغط على المورد الأكثر استهلاكاً وهو (قاعدة البيانات) مما يسرع زمن الاستجابة بشكل ملحوظ.....
- تحديث الموارد Background Refresh قمنا بتطبيق فكرة التحديث الخلفي حيث يحصل المستخدم على البيانات من الكاش فوراً بينما يقوم خيط معالجة منفصل بتحديث الكاش من قاعدة البيانات في الخلفية هذا يضمن بقاء المورد متاحاً دائماً High Availability ويقلل من حدوث الاختناقات عند تحديث البيانات الضخمة.
- تجربة توضح الفرق قبل إدارة الموارد وبعد الإدارة

حسب المعيار العالمي للسيرفرات عالية الأداء، يجب ان يتجاوز زمن استجابة الـ API في الحالات الطبيعية 200ms وفي حالات الضغط العالي يجب ان يتجاوز 500ms لضمان تجربة مستخدم سلسة.

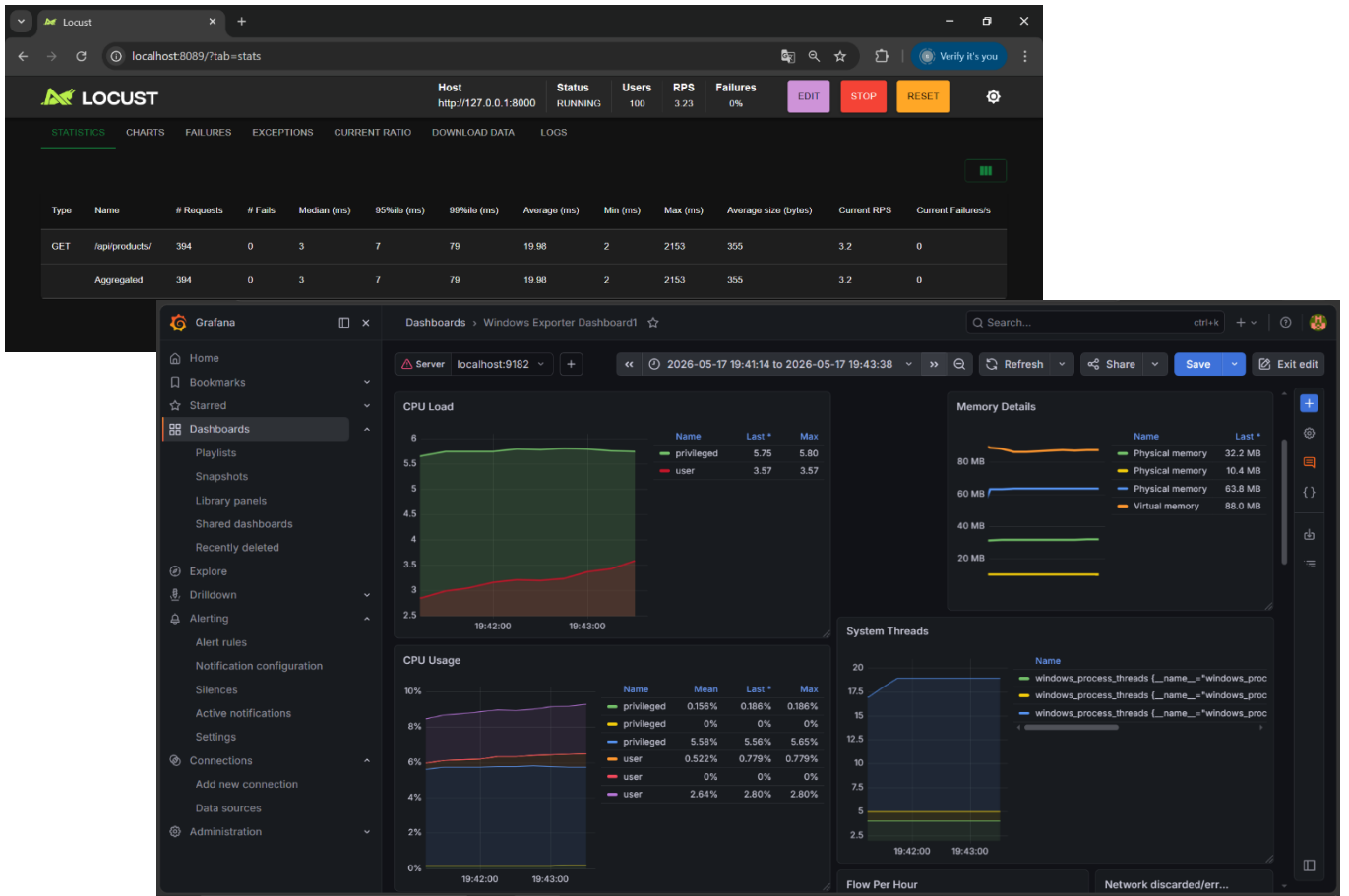
اما بالنسبة لاستهلاك المعالج فالوضع المثالي الوضع المثالي يتراوح بين 30% الى 70% حيث لا يجب ان يتجاوز الـ 80%

اما الذاكرة فان نسبة الاستهلاك المستمر الماثلي تتراوح بين الـ 75% الى 80% حيث قمنا باستخدام مكتبة locust والتي تمكنا من محاكاة سلوك الالف المستخدمين المتزامنين بكفاءة عالية وقياس مدى استجابة النظام تحت الضغوط المختلفة ومن خلال هذه الأداة استطعنا الحصول على تحليل رقمي دقيق للفرق الذي أحدثه تجمع الخيوط Thread Pool وكانت النتائج كالتالي: أولاً:

قمنا بمحاكاة 100 مستخدم يقوم بطلب عرض المنتجات في الوقت نفسه (في نفس الثانية) قبل ادارة الموارد:



بعد إدارة الموارد:

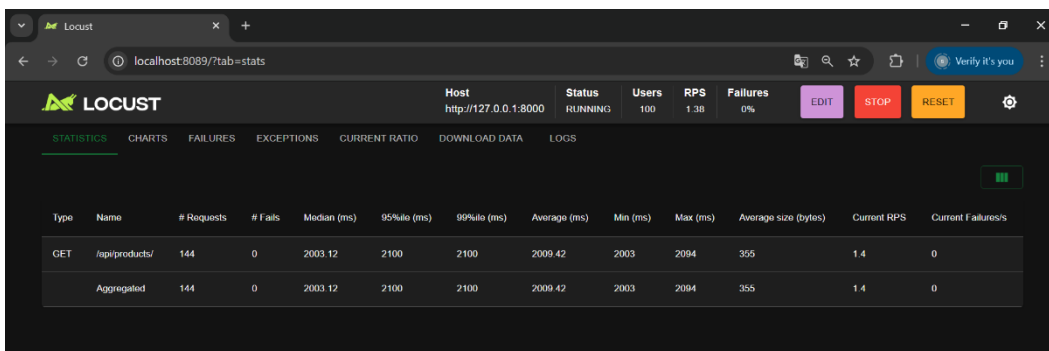


نلاحظ ان الأداء تحسن بشكل واضح

ثانياً:

قمنا بمحاكاة 100 مستخدم يقوم بطلب عرض المنتجات في الوقت نفسه (في نفس الثانية) مع اضافة وقت انتظار ثانيتين (2)time.sleep(2) لمحاكاة الزمن الحقيقي للعمليات الثقيلة (مثل الدفع).

قبل إدارة الموارد:



نلاحظ تحسن الاداء بشكل كبير رغم ان العمليات ثقيلة الا ان معدل الاستجابة 4ms تقريبا واستهلاك المعالج والذاكرة جيد ومستقر.

الطلب الثالث: المعالجة غير المتزامنة Asynchronous Queue:

نقل المهام التي لا يحتاج المستخدم لانتظارها (مثل اصدار فواتير أو ارسال الاشعارات) خارج المسار الرئيسي للطلب.

لفصل وقت الاستجابة للمستخدم عن وقت المعالجة الثقيلة أي بدلا من المعالجة المباشرة عند ارسال طلب للسيرفر ومراجعة قاعدة البيانات وتوليد فاتورة وارسل ايميل ثم الرد على المستخدم (الطلب يسير بمسار واحد) هذا سيجعل المستخدم ان ينتظر كثير أمام الشاشة بدلا من الحصول على طلبه بسرعة، لذلك نجعل العمليات الثقيلة والتي لا يحتاج المستخدم انتظارها للعمل خارج المسار الرئيسي للطلب الأساسي، أي بمجرد استلام الطلب والتأكد من صحته يقوم السيرفر بجعل المهام الثقيلة تعمل على مسار اخر ويرسل الرد فورا للمستخدم.

وكما لاحظنا عند ارسال طلب شراء على الـ postman انتهى الطلب في أجزاء من الثانية وقام بإرسال Accepted 202.

وفي المسار الخلفي بدأ العمل بشكل مستقل لإصدار الفواتير وارسل الاشعارات .

لذلك قمنا باستخدام ThreadPoolExecutor لتنفيذ المهام الثقيلة خارج المسار الرئيسي للطلب أي لتنفيذ المهام بالخلفية (Background Tasks) لبعض العمليات التي لا يحتاج المستخدم انتظارها مثل:

- إرسال إيميل
- إصدار فاتورة

لذلك بدل من تنفيذها داخل نفس الطلب، قمنا بتشغيلها ضمن Threads منفصلة.

```
184         quantity=quantity
185     )
186
187
188     io_executor.submit(send_email_notification, order.id)
189
190
191
192
```

عند تنفيذ انشاء طلب شراء تم نقل مهمة إرسال الإيميل الى Thread منفصل.

```
290
291
292
293     cpu_executor.submit(generate_invoice, order.id)
294
295
296
297
```

وعند تنفيذ عملية الدفع تم نقل عملية توليد الفاتورة الى CPU Thread منفصل.

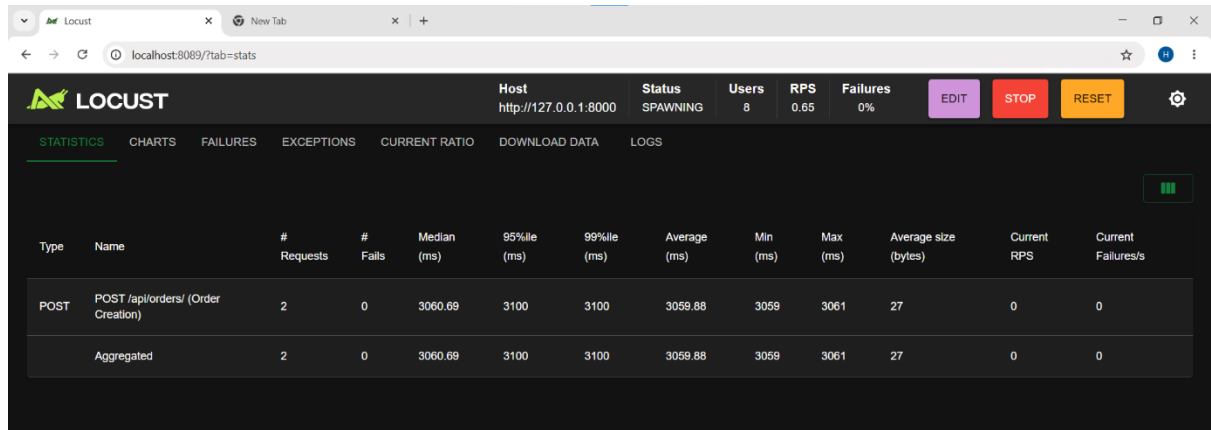
```
[12/May/2026 10:23:21] "POST /api/orders/ HTTP/1.1" 202 38
Running Email Task in thread: IO_Worker_1
Email sent successfully for order 11317
```



```
[12/May/2026 10:26:26] "POST /api/orders/11317/pay/ HTTP/1.1" 202 41
Running Invoice Task in thread: CPU_Worker_0
Invoice generated for order 11317
```

ظهر في التيرمينال أن المهام تعمل ضمن Threads مختلفة وهذا يثبت أن المعالجة تتم في الخلفية وليس داخل المسار الرئيسي للطلب.

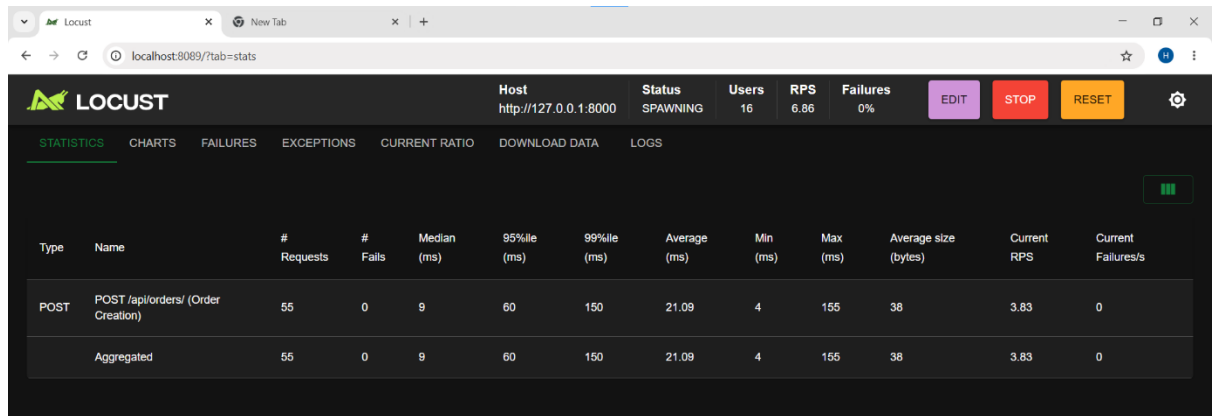
كما قمنا بمحاكاة 50 مستخدم على LOCUST وظهرت النتائج التالية :



1. تم تنفيذ جميع العمليات ضمن نفس المسار الرئيسي للطلب، أي أن السيرفر كان ينتظر انتهاء:

إنشاء الطلب وإصدار الفاتورة وإرسال الإيميل قبل إرسال الاستجابة للمستخدم.

وقد تم اختبار السيرفر باستخدام Locust لمحاكاة 50 مستخدم متزامن فكان متوسط زمن الاستجابة يتجاوز 3 ثواني .



2. فصل العمليات مثل إصدار الفواتير وإرسال الإشعارات عن المسار الرئيسي للطلب باستخدام

ThreadPoolExecutor، حيث أصبح السيرفر يرسل الاستجابة مباشرة للمستخدم ثم يكمل تنفيذ المهام بالخلفية ضمن Threads مستقلة.

وقد تم اختبار السيرفر باستخدام Locust لمحاكاة 50 مستخدم متزامن، فكانت النتائج أفضل بشكل واضح، حيث انخفض متوسط زمن الاستجابة الى حوالي: 11 ms بعد أن كان يتجاوز 3 ثواني في الحالة الأولى..

الطلب الرابع: المعالجة الدفعية المتوازية المجدولة تلقائياً (Parallel Batch Processing & Scheduling):

1. الهدف من الطلب:

تحسين أداء النظام عند التعامل مع كميات ضخمة من البيانات الناتجة عن عمليات الشحن الكثيف (Bulk Charging). يهدف هذا المتطلب الى تجنب معالجة البيانات دفعة واحدة بشكل تسلسلي مما يسبب اختناقاً للمعالج (CPU Bottleneck)، واستبدال ذلك بتقطيع البيانات الى حزم صغيرة (Chunks) وتوزيعها ديناميكياً على خيوط معالجة متعددة (Parallel Workers) تعمل بالتوازي في الخلفية عبر جدول زمني تلقائي (APScheduler).

2. التقنيات والأدوات المستخدمة:

- APScheduler (Advanced Python Scheduler): لجدولة المهمة وتفعيلها تلقائياً بانتظام بدون تدخل برمجي خارجي.
- ThreadPoolExecutor (cpu_executor): مفسر الخيوط التفرعية المخصص للعمليات الحسابية الكثيفة (CPU-bound tasks).
- Django ORM Optimization (select_related): لجلب البيانات المرتبطة من قاعدة البيانات ككتلة واحدة (Eager Loading) لتسريع الاستعلامات وضمان الأمان التفرعي (Thread Safety).

3. الشرح للملف core/tasks.py:

- تم بناء الدالتين أساسيتين لتحقيق هذا المتطلب وفصل المسؤوليات أكاديمياً:
- a. Orchestrator Function (process_daily_sales_chunks): الدالة المنسقة التي تقيس الأداء، تقسم البيانات، وتوزع المهام.
 - b. Orchestrator Function (process_daily_sales_chunks): الدالة المنسقة التي تقيس الأداء، تقسم البيانات، وتوزع المهام.

4. تحليل المخرجات وسلوك النظام بالتيرمينال:

عند تشغيل المجدول وضخ 50,000 سجل عبر الـ BulkDataChargerAPIView نلاحظ التالي:

```
APScheduler started successfully within 'core' app!  
Starting Bulk Data Injection...  
Successfully injected 50000 paid orders into PostgreSQL!  
[18/May/2026 03:41:40] "POST /api/charge-db/ HTTP/1.1" 201 71
```

```
Processing began without partitioning all data (250000 register)...
Processing output without division:
  Number of requests processed: 250000
  Final price of orders (Total Revenue): 50000000.00
  Total time consumed: 11.2513 s
```

```
Processing began by splitting the data into batches: 1000...
```

```
[Parallel Worker] Chunk #1 Finished Processing !
[Parallel Worker] Chunk #2 Finished Processing !
[Parallel Worker] Chunk #5 Finished Processing
[Parallel Worker] Chunk #7 Finished Processing
[Parallel Worker] Chunk #9 Finished Processing
[Parallel Worker] Chunk #13 Finished Processing
[Parallel Worker] Chunk #14 Finished Processing
[Parallel Worker] Chunk #11 Finished Processing
[Parallel Worker] Chunk #15 Finished Processing
```

```
[Parallel Worker] Chunk #161 Finished Processing !
[Parallel Worker] Chunk #164 Finished Processing !
[Parallel Worker] Chunk #200 Finished Processing !
[Parallel Worker] Chunk #166 Finished Processing !
[Parallel Worker] Chunk #162 Finished Processing !
```

```
Processing outputs with segmentation:
```

```
  Number of batches processed (Total Chunks): 250
  Number of requests processed: 250000
  Final price for orders: 50000000.00
  Final time consumed: 0.8135 s
```

```
[BENCHMARK RESULT]
```

```
Batch Processing it was faster by a factor of 10.4377 s
```

```
Batch Benchmark Finished Successfully
```

- تفسير السلوك التفرعي: تظهر العبارة [Parallel Worker] Chunk #x Finished processing بشكل متداخل وغير متتابع في التيرمينال؛ مما يثبت أن خيوط المعالج تعالج مجموعات البيانات بالتوازي (Concurrent Execution).
- نتيجة اختبار الأداء: تظهر مخرجات المقارنة الزمنية تفوق الـ Parallel Batch Processing على المعالجة التقليدية بفارق زمني شاسع، مما يثبت ارتفاع معدل الإنتاجية لـ (System Throughput) مع زيادة حجم البيانات.

الطلب الخامس: توزيع الأحمال وتوسيع النظام (Load Balancing & Distribution):

1. الهدف من الطلب:

محاكاة تخديم الأنظمة الضخمة تحت ضغط الطلبات الكثيف (High Traffic Systems) من خلال تطبيق مفهوم التوسع الأفقي (Horizontal Scaling). يهدف هذا الطلب الى إلغاء فكرة السيرفر الفردي ونقطة الفشل الواحدة (Single Point of Failure)، عبر تشغيل عدّة نسخ مستقلة من المشروع وتوزيع طلبات الزبائن الواردة عليها بالتوازن.

2. التقنيات والأدوات المستخدمة:

- Nginx HTTP Server: يعمل كـ Reverse Proxy وموزع أحمال خارجي حقيقي (Load Balancer).
- Round Robin Algorithm: الخوارزمية السياسية المتبعة في Nginx لتوزيع الطلبات على السيرفرات بالتناوب الدائري المتساوي.
- Multi-Instance Architecture: تشغيل نسختين مستقلتين من تطبيق Django على بورتات مختلفة (8001 و 8000) يتصلان بقاعدة بيانات موحدة PostgreSQL.

3. إعداد خادم وموزع الأحمال (Nginx Configuration):

تم تعديل ملف الإعدادات الخاص بـ Nginx (nginx.conf) ليقوم بالاستماع على البورت الموحد 80 وتميرير الطلبات تفرعياً لمجمع سيرفرات دجانغو المسمى `django_servers`.

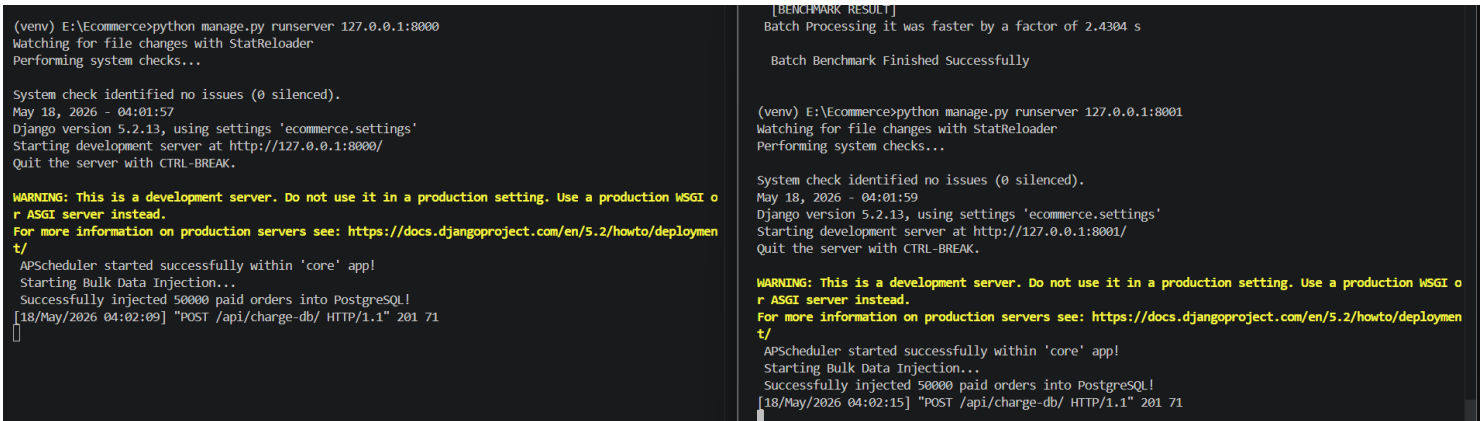
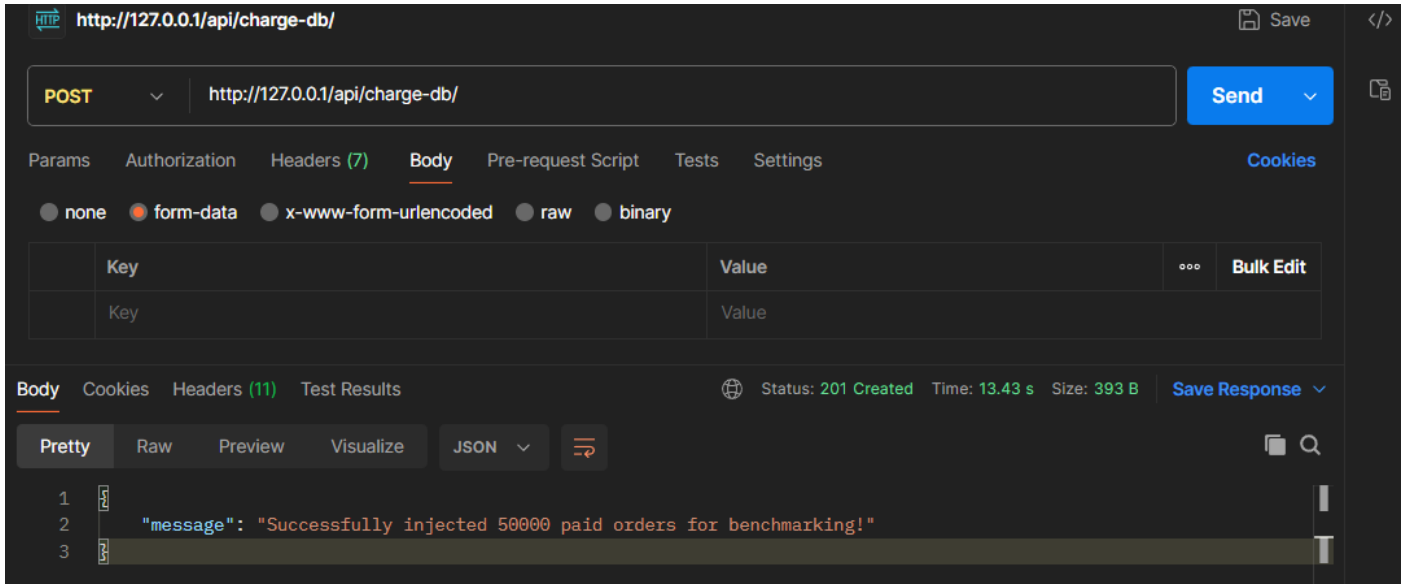
4. طريقة تشغيل واختبار المنظومة الموزعة:

لإثبات تفعيل المنظومة الحقيقية على نظام التشغيل، يتم فتح ثلاث نوافذ أوامر مستقلة:

- النافذة الأولى: تشغيل السيرفر الأول: `python manage.py runserver 127.0.0.1:8000`
- النافذة الثانية: تشغيل السيرفر الثاني: `python manage.py runserver 127.0.0.1:8001`
- النافذة الثالثة: تشغيل موزع الأحمال الخارجي: `path of "nginx.exe" file`

5. تحليل المخرجات من Postman وTerminal:

عند إرسال طلبات دفع متتالية من برنامج Postman الى الرابط الموحد الموجه للبورت 80: GET `http://127.0.0.1/products` نلاحظ السلوك الهندسي التالي:



a. تفسير سلوك خوارزمية الـ Round Robin: عند إرسال طلبات متتالية، يستقبل Nginx الطلبات ويوجهها كالتالي:

- الطلب الأول والثالث تظهر خطوط تسجيل استلامهما (HTTP Logs) في تيرمينال السيرفر 8000.
- الطلب الثاني والرابع تظهر خطوط تسجيل استلامهما فوراً في تيرمينال السيرفر 8001.

b. هذا التناوب الدائري المطلق يؤكد نجاح موازنة الأحمال وتقسيم جهد المعالجة وحقن المفهوم الحقيقي للأنظمة الموزعة (Distributed Systems).

 LOCUST

Host

http://127.0.0.1/

Status

RUNNING

Users

50

RPS

30.86

Failures

0%

EDIT

STOP

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS



الطلب السادس: استراتيجية التخزين المؤقت (Distributed Caching):

تم استخدام Caching لتقليل عدد الاستعلامات المرسلة إلى قاعدة البيانات ، مما يؤدي إلى تقليل زمن الاستجابة وتحسين أداء النظام عند زيادة عدد المستخدمين المتزامنين.

تم تطبيق الـ Cache على البيانات الأكثر استخداماً وهي:

- ✓ قائمة المنتجات .
- ✓ قائمة المنتجات الأكثر مبيعاً .
- ✓ تفاصيل المنتج .

قمنا باستخدام Redis عبر Memurai Developer Edition كطبقة تخزين مؤقت ، تم دمج Redis مع Django باستخدام django.core.cache بحيث يتم تخزين البيانات الأكثر استخداماً في الذاكرة، مما يسمح بإرجاعها بسرعة دون الحاجة إلى تنفيذ استعلام جديد على قاعدة البيانات في كل مرة.

```
16
17
18 CACHES = {
19     "default": {
20         "BACKEND": "django_redis.cache.RedisCache",
21         "LOCATION": "redis://127.0.0.1:6379/1",
22         "OPTIONS": {
23             "CLIENT_CLASS": "django_redis.client.DefaultClient",
24         },
25         "TIMEOUT": 60
26     }
27 }
28
29
30
```

اعتمادنا نمط Cache-Aside ، والذي يعمل بالشكل التالي:

- عند تنفيذ طلب GET يحاول النظام قراءة البيانات من الكاش أولاً.
- إذا لم تكن موجودة (Cache MISS) يتم جلبها من قاعدة البيانات ثم تخزينها في الكاش لفترة زمنية محددة قبل إرجاعها للمستخدم.

X-Cache ⓘ	MISS
-----------	------

- إذا كانت البيانات موجودة (Cache HIT) يتم إرجاعها مباشرة دون الوصول إلى قاعدة البيانات.

X-Cache ⓘ	HIT
-----------	-----

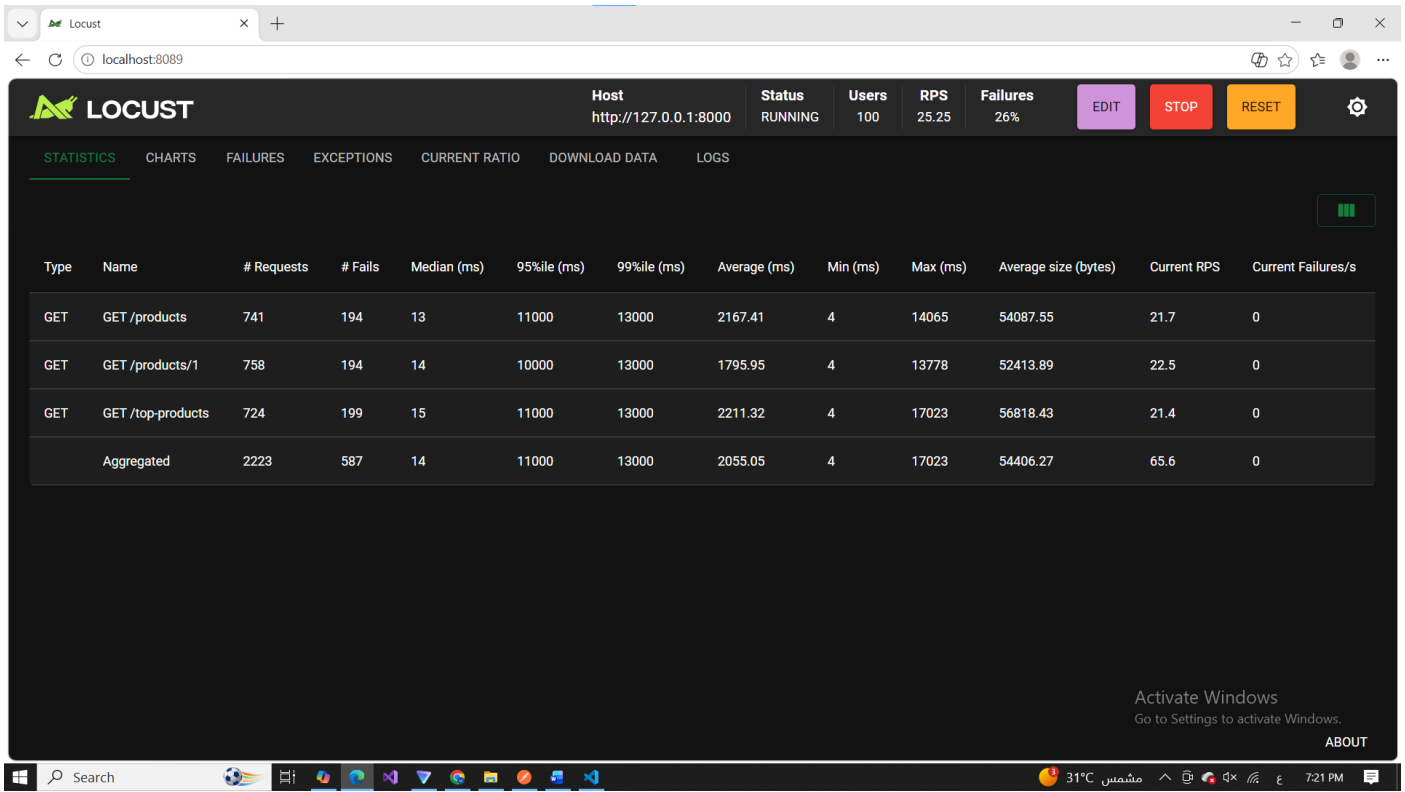
- عند تنفيذ عمليات POST أو PUT أو DELETE يتم حذف مفاتيح الكاش المرتبطة بالبيانات المعدلة (Cache Invalidation) لضمان عدم عرض بيانات قديمة للمستخدمين.

```
C:\Program Files\Memurai\memurai-cli.exe
127.0.0.1:6379> SELECT 1
OK
127.0.0.1:6379[1]> KEYS *
1) ":1:product:1"
2) ":1:products:top10"
3) ":1:products:list"
127.0.0.1:6379[1]>
```

```
C:\Program Files\Memurai\memurai-cli.exe
127.0.0.1:6379> KEYS *
(empty array)
127.0.0.1:6379>
```

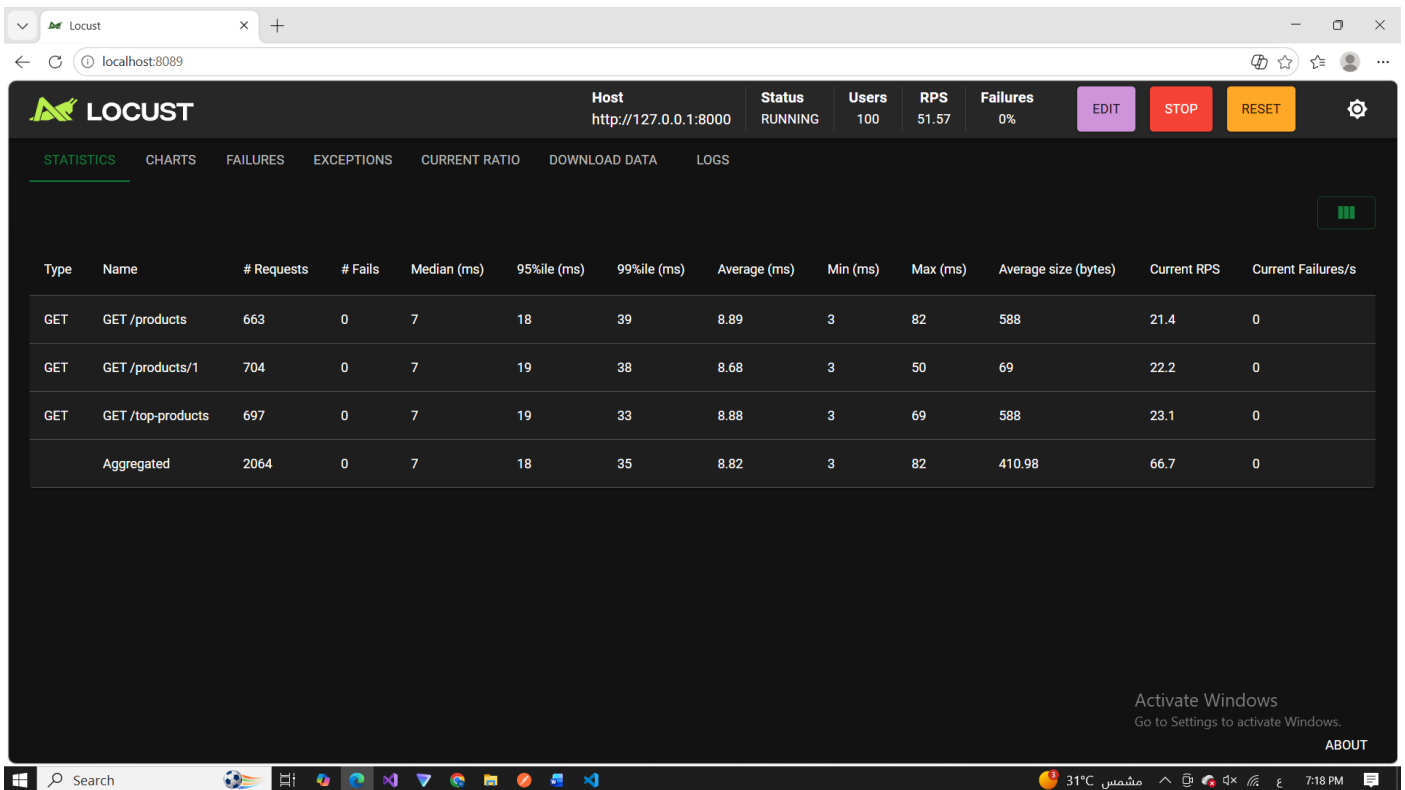
تم استخدام أداة Locust لمحاكاة 100 مستخدم متزامن وإرسال طلبات لطلب قائمة المنتجات و قائمة أكثر المنتجات مبيعا و تفاصيل منتج . فكانت النتائج كالتالي وذلك بدون استخدام الكاش .

قبل استخدام الـ Cache كان كل طلب (Request) يؤدي إلى تنفيذ استعلام جديد على قاعدة البيانات حتى وإن كانت البيانات نفسها قد طُلبت قبل لحظات، مما يزيد الحمل على قاعدة البيانات ويرفع زمن الاستجابة عند زيادة عدد المستخدمين.



تم استخدام أداة Locust لمحاكاة 100 مستخدم متزامن وإرسال طلبات لطلب قائمة المنتجات و قائمة أكثر المنتجات مبيعا و تفاصيل منتج . فكانت النتائج كالتالي وذلك مع استخدام الكاش:

بعد تطبيق الـ Cache أصبحت البيانات تُحفظ مؤقتاً في الذاكرة لمدة محددة، لذلك فإن الطلب الأول فقط يقوم بقراءة البيانات من قاعدة البيانات، بينما يتم تلبية الطلبات التالية مباشرة من الـ Cache دون تنفيذ استعلامات إضافية.



نلاحظ أن :

أظهرت نتائج الاختبار انخفاضاً واضحاً في زمن الاستجابة بعد تطبيق الـ Cache حيث انخفض 99%، كما تضاعف عدد الطلبات التي استطاع الخادم معالجتها في الثانية، و أصبحت نسبة الخطأ معدومة وانخفض عدد الاستعلامات المنفذة على قاعدة البيانات، مما يؤكد فعالية استخدام الـ Caching في تحسين الأداء.

الطلب السابع: التحكم في الأقفال (Concurrency Control):

من أجل حماية البيانات من التعارضات الناتجة عن تنفيذ عدة طلبات في الوقت نفسه قمنا باستخدام أكثر من آلية للترامن لضمان سلامة البيانات ومنع حدوث Race Conditions .

تم تطبيق القفل التشاؤمي (Pessimistic Locking) باستخدام:

```
486
487     def run_cancel_logic():
488
489         with transaction.atomic():
490             order_bg = Order.objects.get(id=id)
491             item = order_bg.orderitem_set.select_for_update().first()
492             if item:
493                 product = item.product
494
```

حيث يتم قفل السجل المطلوب تعديله داخل قاعدة البيانات طوال فترة تنفيذ العملية، مما يمنع أي عملية أخرى من تعديل نفس البيانات حتى انتهاء العملية الحالية.

تم استخدام هذه الآلية في العمليات التالية:

- تعديل الطلب (Update Order)
- حذف الطلب (Delete Order)
- إلغاء الطلب (Cancel Order)

يساعد هذا القفل على منع تعارض العمليات المتزامنة التي قد تؤدي إلى تعديل أو حذف نفس البيانات في الوقت نفسه.

تم تطبيق Distributed Lock باستخدام :

Redis لتنظيم الوصول إلى العمليات الحساسة على مستوى التطبيق، وليس فقط على مستوى قاعدة البيانات.

تم إنشاء قفل فريد لكل عملية اعتماداً على معرف الطلب أو المنتج، بحيث لا يسمح إلا لخيط (Thread) واحد بتنفيذ العملية في الوقت نفسه، بينما تنتظر بقية الطلبات حتى يتم تحرير القفل.

تم استخدام Distributed Lock في أكثر من مكان :

- إنشاء الطلبات (Create Order)
- تعديل الطلبات (Update Order)
- حذف الطلبات (Delete Order)
- إلغاء الطلبات (Cancel Order)

```
302
303 def run_create_order_logic(user):
304     lock_key = f"lock:product:{product_id}"
305
306     print("TRY LOCK:", product_id)
307     acquired = cache.add(lock_key, "1", timeout=10)
308     print("LOCK RESULT:", acquired)
309
310     if not acquired:
311         print("Product is locked by another request")
312         return
313
```

الطلب الثامن : سلامة المعاملات ACID / Integrity Transaction

لضمان تنفيذ العمليات التجارية الحساسة مثل (إتمام عملية الشراء) بشكل كامل أو عدم تنفيذها إطلاقاً، حتى في بيئة تحتوي على عدد كبير من الطلبات المتزامنة

يؤدي إلى ضمان:

- عدم فقدان البيانات
- منع تضارب عمليات الشراء
- الحفاظ على اتساق المخزون والطلبات

تم اعتماد عملية (Checkout إتمام الطلب) كنقطة تطبيق رئيسية لمفهوم ACID ، حيث تشمل العملية عدة خطوات مترابطة:

- التحقق من توفر الكمية في المخزون
- خصم الكمية من المنتج

- إنشاء طلب جديد (Order)
 - إنشاء عناصر الطلب (OrderItem)
 - تنفيذ عملية الدفع (محاكاة)
 - تحديث الحالة إلى PAID
- كل هذه الخطوات يجب أن تنفذ كوحدة واحدة متكاملة.

تم استخدام:

- transaction.atomic() لضمان تنفيذ العملية كوحدة واحدة
- F Expressions لتحديث المخزون بشكل آمن تحت الضغط المتزامن
- Distributed Lock (cache.add) لمنع race condition بين الطلبات
- ThreadPoolExecutor لتنفيذ العمليات غير المتزامنة خارج المسار الرئيسي

خصائص ACID المطبقة

1. Atomicity

إما أن تنجح كل خطوات الطلب أو تفشل كلها باستخدام transaction.atomic().

2. Consistency

يتم الحفاظ على صحة البيانات (المخزون لا يصبح سالبًا).

3. Isolation

تم استخدام:

- Distributed Lock
- select_for_update في بعض العمليات

لمنع تعارض الطلبات.

4. Durability

بعد نجاح العملية، يتم حفظ البيانات في قاعدة PostgreSQL بشكل دائم.

النتائج

- منع تضارب عمليات الشراء عند الضغط العالي
- الحفاظ على دقة المخزون

- تنفيذ العمليات بشكل آمن تحت التزامن
- تقليل أخطاء race condition بشكل كبير

الطلب التاسع: اختبار الاستقرار تحت الضغط (Stress Testing)

تم تنفيذ اختبار استقرار للنظام بهدف قياس قدرة منصة التجارة الإلكترونية على التعامل مع عدد كبير من المستخدمين المتزامنين دون انهيار أو فقدان بيانات، وذلك باستخدام أداة **Locust** لمحاكاة الضغط الحقيقي على النظام

تم تصميم سيناريو يحاكي مستخدمين حقيقيين يقومون بالعمليات التالية:

- تسجيل حساب جديد
- تسجيل الدخول والحصول على Token
- عرض المنتجات
- عرض المنتجات الأكثر مبيعاً
- تنفيذ عمليات Checkout
- إنشاء طلبات (Orders)

تم توزيع الأوزان بين العمليات لمحاكاة سلوك المستخدم الحقيقي.

تم استخدام Queue تحتوي على 500 حساب مختلف، حيث يحصل كل مستخدم وهمي على حساب فريد عند بدء التشغيل. وفي حال فشل التسجيل أو تسجيل الدخول يتم إعادة الحساب إلى الطابور للاستفادة منه لاحقاً، مما يمنع فقدان الحسابات أثناء الاختبار ويحافظ على استمرارية الضغط.

```
from locust import HttpUser, between, task
import gevent
user_queue = Queue()

for i in range(1, 500):
    user_queue.put((f"user_saratest12_{i}", "testpass123"))
```

الهدف:

- ضمان أن كل مستخدم وهمي يمتلك حساب مستقل
- منع مشاركة نفس الحساب بين عدة مستخدمين متزامنين
- تحسين دقة نتائج اختبار الضغط

تم تطوير آلية تسجيل + تسجيل دخول تتضمن:

- إعادة المحاولة (Retry Mechanism)
- تأخير بسيط لضمان تهيئة الحساب
- التعامل مع حالات الفشل بشكل مرن

تم نقل عملية التحقق من المستخدم إلى `on_start()` بحيث يتم إنشاء الحساب وتسجيل الدخول مرة واحدة فقط عند بدء المستخدم الوهمي، بدلاً من إعادة التحقق قبل كل مهمة. أدى ذلك إلى تقليل الحمل الناتج عن عمليات المصادقة المتكررة وجعل الاختبار أقرب إلى سلوك المستخدم الحقيقي.

```
gevent.sleep(0.2)

max_retries = 3
for attempt in range(max_retries):
    with self.client.post(
        "/api/login/",
        json={"username": username, "password": password},
        catch_response=True,
    ) as login_response:
        if login_response.status_code == 200:
            token = login_response.json().get("access")
            self.headers = {"Authorization": f"Bearer {token}"}
            login_response.success()
            self.authenticated = True
            print(f"{attempt + 1} في المحاولة {username}: تم دخول المستخدم بنجاح")
            break
        else:
```

استخدام `gevent.sleep` يسمح للمستخدمين الآخرين بالمتابعة أثناء الانتظار دون حجز Thread كامل، مما يزيد من كفاءة اختبار الضغط ويعطي نتائج أقرب للواقع.

نتائج الاختبار (الاستنتاج العام)

- النظام استطاع التعامل مع عدد كبير من المستخدمين المتزامنين
- عمليات القراءة (Products / Top Products) كانت مستقرة بفضل الكاش
- عمليات Checkout كانت الأثقل لكنها بقيت مستقرة
- تم تحقيق أداء مقبول تحت الضغط دون انهيار النظام



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/checkout/	240	0	4	15	40	5.79	2	43	41	11.3	0
POST	/api/login/	100	0	16	65	170	23.25	3	169	494	0	0
POST	/api/orders/	131	0	4	16	22	5.49	2	46	38	7	0
GET	/api/products/	394	0	3	18	180	6.95	1	216	140	18.6	0
POST	/api/register/	100	0	31	300	350	58.63	15	351	63	0	0
GET	/api/top-products/	228	0	3	19	30	5.74	1	119	140	12.1	0
Aggregated		1193	0	4	39	210	12.02	1	351	132.1	49	0

الطلب العاشر: تحليل الاختناقات (Bottleneck Analysis & Benchmarking)

تم تحليل أداء النظام لتحديد نقاط الاختناق (Bottlenecks) وقياس الأداء قبل وبعد تطبيق التحسينات، بهدف تحسين استجابة النظام تحت الضغط العالي.

تحسين إدارة اتصال قاعدة البيانات:

CONN_MAX_AGE': 600'

الهدف:

- إبقاء اتصال قاعدة البيانات مفتوح لمدة 10 دقائق
- تقليل تكلفة فتح وإغلاق الاتصالات المتكرر
- تحسين الأداء تحت الضغط العالي

تحسين Redis Cache

تم تحديد حد أقصى للاتصالات:

max_connections": 1000"

الهدف:

- منع استهلاك زائد للموارد
- تجنب اختناق الاتصالات (Connection Bottleneck)

تحسين المصادقة (Authentication Speed)

تم استخدام Hash سريع مؤقتًا أثناء الاختبار:

```

7 PASSWORD_HASHERS = [
8     'django.contrib.auth.hashers.MD5PasswordHasher',
9 ]
0

```

الهدف:

- تقليل وقت تسجيل الدخول
- تسريع اختبارات الضغط
- تقليل الحمل على CPU

تحسين الكاش في النظام

تم اعتماد استراتيجية:

- حذف الكاش بعد أي تعديل (Update/Delete/Order)
- ضمان أن البيانات المعروضة حديثة دائمًا

```

cache.delete("products:list")
cache.delete(f"product:{product_id}")

```

الهدف:

- منع تقديم بيانات قديمة
- تحسين اتساق البيانات (Consistency)

تحسين السيرفر (Deployment Layer)

تم استبدال Django Development Server (runserver) بخادم Waitress المخصص للإنتاج.

تم إنشاء ملف تشغيل مستقل يقوم بتهيئة الخادم بالقيم التالية:

```

# Terminal تفعيل طباعة سجلات السيرفر في الـ
logger = logging.getLogger('waitress')
logger.setLevel(logging.INFO)
ch = logging.StreamHandler()
ch.setLevel(logging.INFO)
logger.addHandler(ch)

print("=== Starting Waitress Server on http://127.0.0.1:8000 ===")

serve(
    application,
    host='127.0.0.1',
    port=8000,
    threads=30,
    connection_limit=300
)

```

السبب:

- runserver غير مناسب للضغط العالي
- ينهار عند عدد كبير من الطلبات
- غير مصمم للـ concurrency الحقيقي

حيث تم تخصيص:

- 30 Thread لمعالجة الطلبات المتزامنة .
 - حد أقصى 300 اتصال مفتوح في نفس الوقت .
 - إدارة منظمة للطلبات عبر Thread Pool داخلي .
- وقد أدى ذلك إلى منع انهيار الخادم أثناء الضغط المرتفع وتحسين استقرار النظام بشكل ملحوظ مقارنةً بـ runserver.

مميزات Waitress:

- يعتمد Thread Pool Management
- يحتوي على Queue داخلية للطلبات
- يوزع الحمل بشكل منظم
- يمنع انهيار السيرفر تحت الضغط

نتائج التحسين:

قبل التحسين:

- بطء في تسجيل الدخول
- انهيار عند ضغط عالي في Locust
- فقدان بعض الطلبات

• استهلاك عالي للموارد

 LOCUST

Host

http://127.0.0.1:8000

Status

STOPPED

RPS

35.76

Failures

2%

NEW

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

 LOGS



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/checkout/	854	45	7	190	420	32.34	2	1031	10178.39	11.6	1.3
POST	/api/login/	100	1	25000	29000	30000	25004.14	6979	29754	2913.76	0	0
POST	/api/orders/	437	18	7	120	460	26.77	2	1130	6295.44	6.6	0.6
GET	/api/products/	1231	0	5	41	300	16.27	1	1349	140.5	19	0
GET	/api/top-products/	878	0	5	45	290	17.76	1	1388	140.54	12.3	0
	Aggregated	3500	64	6	270	27000	735.82	1	29754	3437.48	49.5	1.9

بعد التحسين:

بعد تطبيق التحسينات على طبقة الخادم (Waitress) ، وإدارة الاتصالات، وتحسين الكاش، وتحسين سيناريو Locust ، انخفضت نسبة الأخطاء إلى 0% أثناء الاختبارات النهائية، مع المحافظة على استقرار النظام تحت حمل مرتفع من المستخدمين المتزامنين.

 LOCUST

Host
http://127.0.0.1:8000

Status
STOPPED

RPS
49.42

Failures
0%

NEW

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/checkout/	240	0	4	15	40	5.79	2	43	41	11.3	0
POST	/api/login/	100	0	16	65	170	23.25	3	169	494	0	0
POST	/api/orders/	131	0	4	16	22	5.49	2	46	38	7	0
GET	/api/products/	394	0	3	18	180	6.95	1	216	140	18.6	0
POST	/api/register/	100	0	31	300	350	58.63	15	351	63	0	0
GET	/api/top-products/	228	0	3	19	30	5.74	1	119	140	12.1	0
	Aggregated	1193	0	4	39	210	12.02	1	351	132.1	49	0

مراقبة الأداء وجمع القياسات

تم دمج مكتبة Django Prometheus داخل النظام:

والتي تقوم بجمع مؤشرات الأداء الخاصة بالنظام مثل:

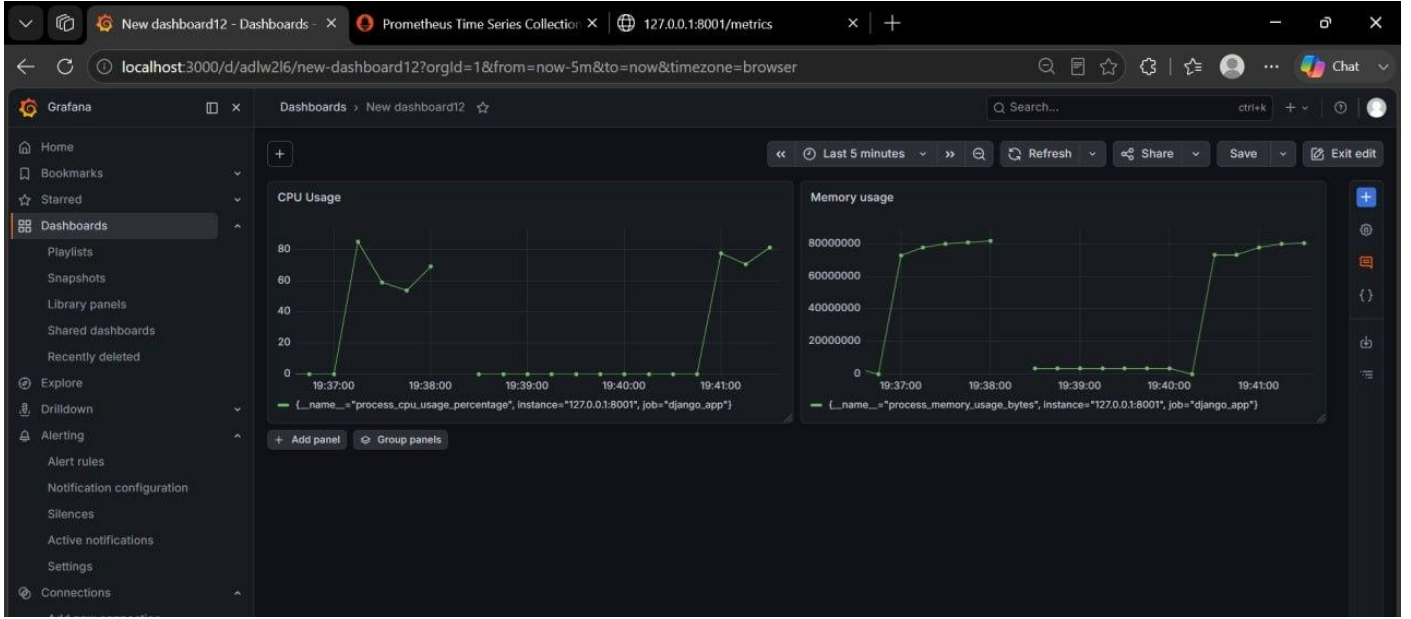
- عدد الطلبات المنفذة .
- أزمنة الاستجابة .
- عدد الأخطاء .

• استهلاك الموارد .

وقد ساعدت هذه المؤشرات في مراقبة أداء النظام أثناء اختبارات الضغط وتحديد نقاط الاختناق المحتملة.

مراقبة استهلاك الموارد (CPU & Memory Monitoring)

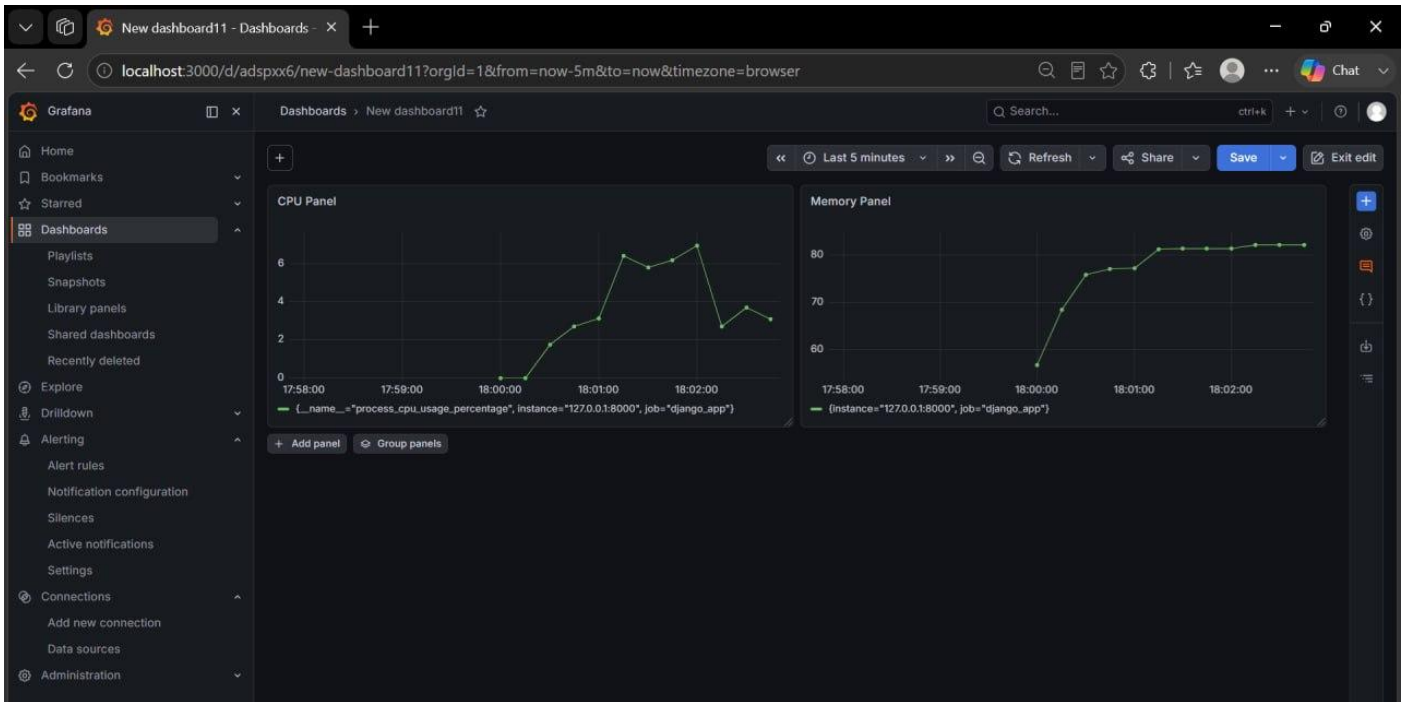
خلال تنفيذ اختبارات الضغط باستخدام Locust على النسخة الأولى من النظام، تم مراقبة استهلاك موارد الخادم لتقييم أداء النظام تحت الأحمال المرتفعة قبل تطبيق أي تحسينات. أظهرت النتائج أن النظام كان أقل كفاءة في التعامل مع عدد الطلبات المتزامنة، مما انعكس على استهلاك الموارد.



تظهر الصورة:

- ارتفاع استهلاك المعالج (CPU Usage) بشكل ملحوظ مقارنة، مع تذبذب واضح أثناء تنفيذ الطلبات .
- استهلاك الذاكرة (Memory Usage) كان أعلى نسبياً وظهر عليه تغير مستمر خلال الاختبار .
- وجود تقلبات في الأداء (Fluctuations) مما يدل على عدم استقرار النظام تحت الضغط .
- لا يوجد استقرار ثابت في استهلاك الموارد عند زيادة الحمل.

خلال تنفيذ اختبارات الضغط باستخدام Locust بعد التحسين ، تمت مراقبة استهلاك موارد الخادم بشكل مستمر بهدف تقييم استقرار النظام تحت الأحمال المرتفعة. تم التركيز على مؤشرين رئيسيين هما استهلاك المعالج (CPU Usage) واستهلاك الذاكرة (Memory Usage) ، حيث تعكسان قدرة النظام على إدارة الموارد بكفاءة أثناء معالجة عدد كبير من الطلبات المتزامنة



تظهر الصورة:

استهلاك **CPU** بقي منخفضاً نسبياً (تقريباً بين 2% و 7%) بالمقارنة مع الحالة قبل التحسين .

استهلاك الذاكرة (**Memory**) استقر حول 80 MB تقريباً بعد بدء الاختبار .

لا توجد قفزات حادة أو استهلاك غير طبيعي للموارد.